

Pràctica:

1. Introducció

L'objectiu d'aquesta pràctica és desenvolupar una aplicació web funcional per a la gestió d'una biblioteca, aplicant conceptes de Programació Orientada a Objectes (POO), manipulació avançada del DOM, validacions mitjançant expressions regulars (RegExp), persistència en LocalStorage i consum de dades asíncrones amb la Fetch API.

2. Estructura HTML (Base)

Utilitza la següent estructura mínima per al fitxer index.html. Pots afegir CSS per millorar la interfície, però no has de modificar els id proporcionats:

HTML

```
<h1>Gestió de Biblioteca</h1>

<section id="seccio-registre">
  <form id="formLibro">
    <input id="titulo" placeholder="Títol" required>
    <input id="autor" placeholder="Autor" required>
    <input id="isbn" placeholder="ISBN (Format: 000-0-000)">
    <input id="anio" placeholder="Any de publicació">
    <button type="submit">Afegir llibre</button>
  </form>
</section>

<hr>

<section id="seccio-busqueda">
  <input id="busqueda" placeholder="Cercar per títol o autor...">
  <div id="stats"></div> </section>

<ul id="listaLibros"></ul>

<section id="acciones-globales">
  <button id="btnRanking">Veure Rànquing Préstecs</button>
  <button id="btnGuardar">Guardar a LocalStorage</button>
  <button id="btnCargar">Recuperar de LocalStorage</button>
  <button id="btnFetch">Importar Llibres (JSON)</button>
</section>
```

3. Requisits del Llenguatge (JavaScript)

3.1. Model de Dades (POO)

Crea una classe anomenada Libro que inclogui:

- **Propietats:** titulo, autor, isbn, any i prestamos (inicialitzat a 0).
- **Mètodes:** * prestar(): Incrementa en 1 el comptador de préstecs.

Nota: Inicialitza l'aplicació amb un array d'almenys **4 instàncies** de la classe Libro.

3.2. Lògica de Funcions Obligatòries

Implementa la lògica necessària assegurant l'ús dels mètodes d'Array especificats:

1. **listarLibros()**: Renderitza l'array en el ul#listaLibros. Cada element ha d'incloure botons per a "Eliminar" i "Prestar". (Ús de forEach).
2. **buscarLibro(titulo)**: Localitza un objecte llibre per coincidència exacta. (Ús de find).
3. **eliminarLibro(isbn)**: Elimina el llibre de l'array principal. (Ús de findIndex).
4. **filtrarLibros(texto)**: Filtra la llista dinàmicament mentre l'usuari escriu al cercador. (Ús de filter i includes).
5. **rankingPrestamos()**: Reordena la llista de major a menor nombre de préstecs. (Ús obligatori de sort() sobre una còpia de l'array).
6. **estadisticas()**: Calcula i mostra al DOM: total de llibres, suma de préstecs, any més antic i més recent. (Ús obligatori de reduce).

3.3. Validacions i Seguretat

Abans d'afegir un llibre a l'array, valida:

- Camps obligatoris: Títol i Autor.
- Any: Ha de ser un valor numèric.
- ISBN: Ha de complir el patró `/^[0-9]{3}-[0-9]{1,5}-[0-9]{1,7}$/` . (Fes servir el mètode test()).

4. Persistència i Asincronia

4.1. Web Storage

Implementa la funcionalitat dels botons corresponents perquè les dades sobrevisquin al refresc de la pàgina:

- Desa les dades utilitzant `localStorage.setItem()` i `JSON.stringify()`.
- Recupera les dades amb `localStorage.getItem()` i `JSON.parse()`.

4.2. Fetch API

Crea una funció asíncrona `cargarLibrosExternos()` que consumeixi un fitxer local `libros.json`:

- Ha de gestionar errors mitjançant un bloc `try...catch`.
- Comprova que la resposta és correcta amb la propietat `ok`.
- Els llibres importats s'han de convertir en instàncies de la classe `Libro`.

5. Criteris d'Avaluació (Checklist)

Per superar la pràctica, el codi ha d'incloure explícitament:

- ☐ Ús d'**Arrow Functions** i **Template Literals**.
- ☐ Implementació de `addEventListener` per a tots els esdeveniments (evitar `onclick` a l'HTML).
- ☐ Ús dels mètodes d'Array: `forEach`, `find`, `findIndex`, `filter`, `sort`, `reduce`, `map` i `includes`.
- ☐ Gestió correcta de l'estat: si s'elimina un llibre, la llista del DOM s'ha d'actualitzar.
- ☐ Codi net, modular i comentat.

Instrucció de lliurament: Comprimeix en un fitxer `.zip` el teu HTML, CSS i JS. Si has creat el fitxer `libros.json` per al Fetch, inclou-lo també.